PostFinance

Migrating a Java App to Go

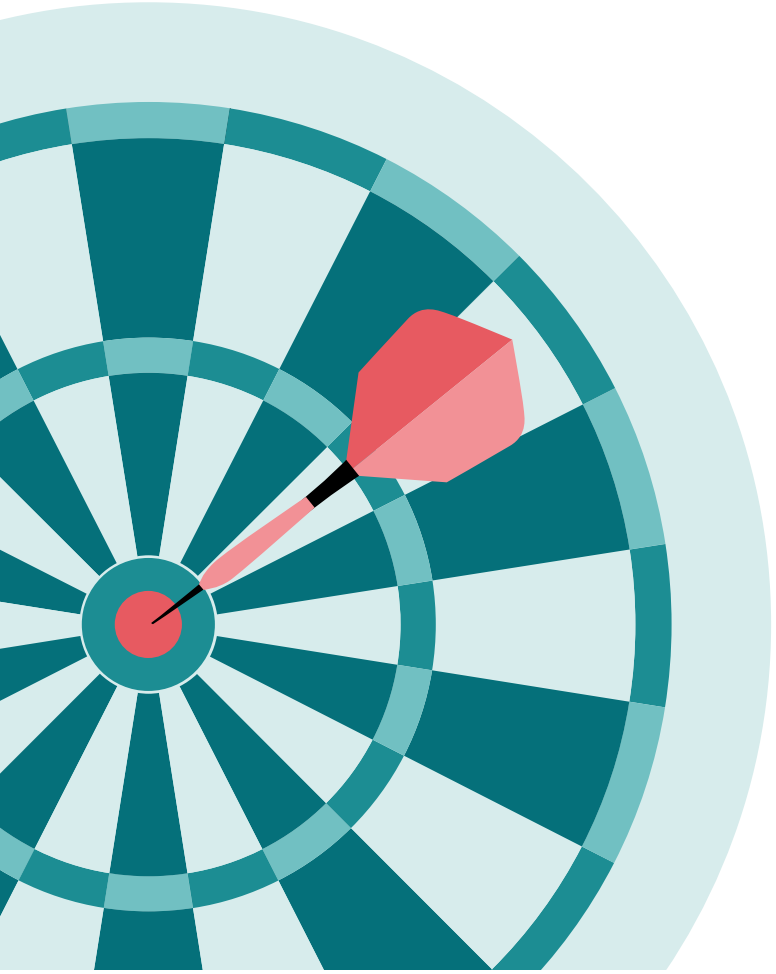
Lessons from a Go Newbie

Bärner Go Meetup Bern, 2026-11-05

Roger Brechbühl, PostFinance AG



Objectives



You will …

- get Golang information on beginner level
- see some differences between Java and Golang
- have some topics to discuss at the Apéro
- be hopefully encouraged for talking at the next Go meetup

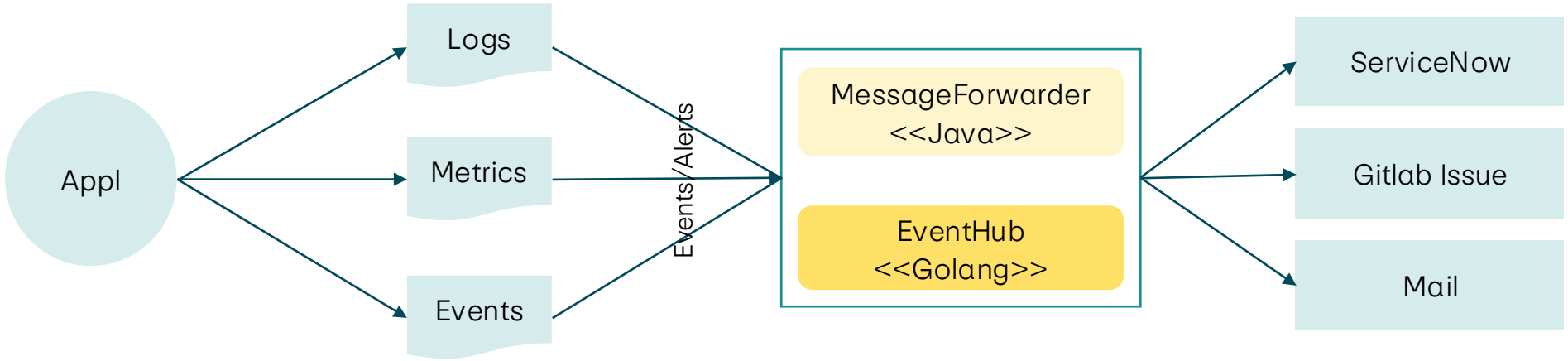


Introduction – “Foreword”

- No Java-bashing
- I’m not up-to-date with development with the latest Java ecosystem
- I’m a Go newbie
- A lot of my statements would most probably be different on a larger code base or with more experience
- Content marked with * is AI generated (Github and Microsoft Copilot)

Introduction – “About me”

- Roger Brechbühl, Platform Engineer at PostFinance
- Work in the Platform and Infrastructure team (Platin)
  Linux/Windows, Kubernetes, Cloud, Databases, Middleware, Event Streaming, Virtualization, Storage, Backup, Observability, Networking etc
- My Area: Observability based on OpenTelemetry, including Alerting
  Victoria Metrics, Grafana Tempo, Splunk, Grafana UI, Opentelemetry Collector etc

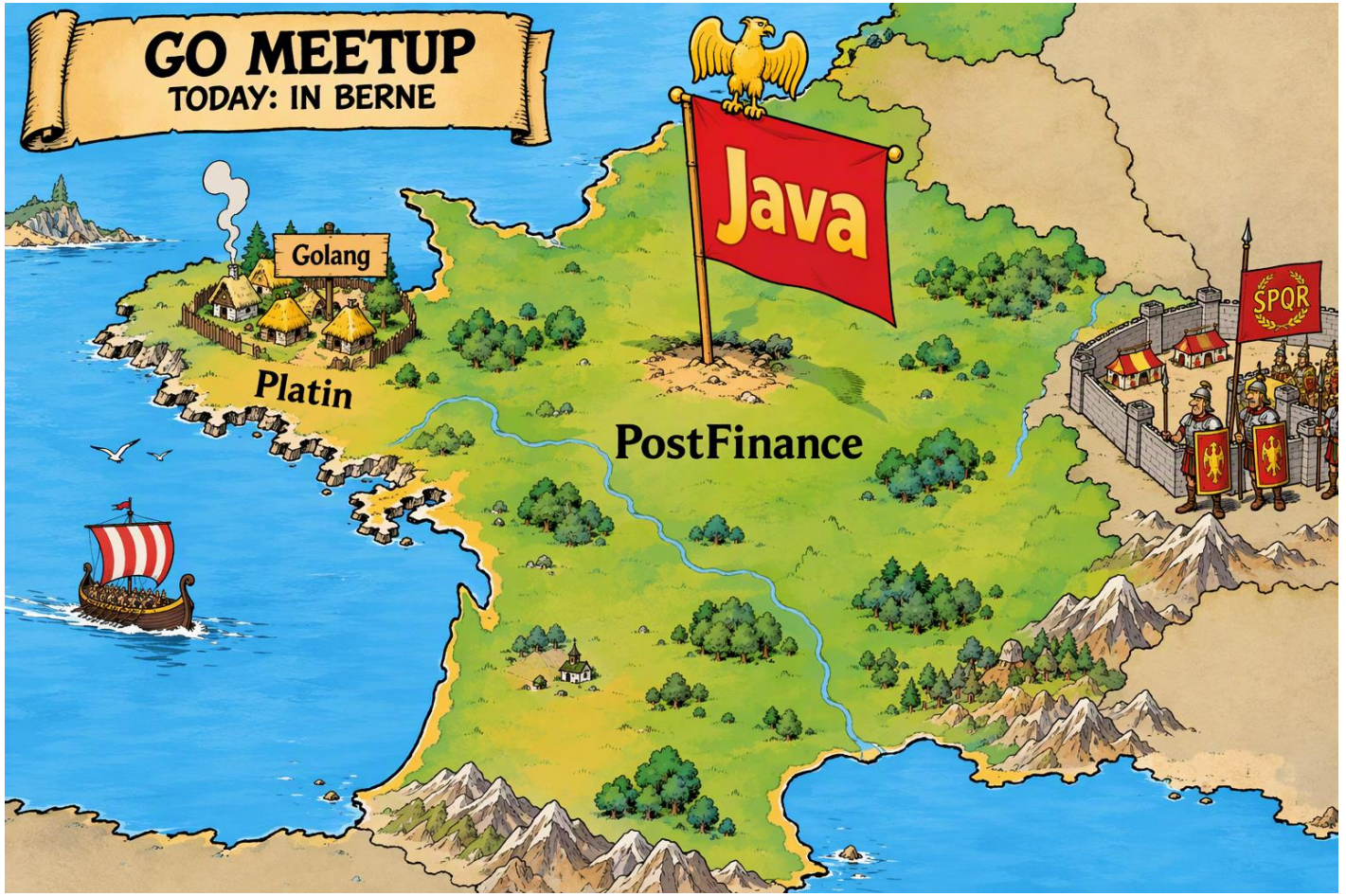


Introduction– “Why golang?”

Platform-Engineering at PostFinance

- Use existing (OpenSource) products
  Kubernetes, Victoria Metrics, Gitlab,...
- Provide Self Services to order infrastructure resources
  Order k8s namespaces via Gitlab Repo
- Automate everything
  namespace-creator: create namespace, prepare Vault, update inventory

  discovery: register metric endpoints to be scraped by VictoriaMetrics



Introduction– “Why migrating?”

Some maintenance is obviously necessary, and some updates look like a rabbit hole



Golang – “Toolchain”

The go binary contains all the basics which are needed for developing with go, an excerpt of commands (`go help`):

- `build` compile packages and dependencies
- `doc` show documentation for package or symbol
- `env` print Go environment information
- `fmt` gofmt (reformat) package sources
- `get` add dependencies to current module and install them
- `install` compile and install packages and dependencies
- `mod` module maintenance
- `run` compile and run Go program
- `test` test packages
- `vet` report likely mistakes in packages

Always the same command, not *java*, *javac*, *jar*, *javadoc*

No Junit, no Ant/Maven, no Checkstyle, no SonarQube etc needed!

```
# go install -v
github.com/golangci/golangci-
lint/v2/cmd/golangci-lint@v2.15.0
```

Downloads, builds and installs *golangci-lint* in $GOPATH/bin

And by the way, with

```
# go env
GOPATH='/Users/brechbuehr/go'
GOPROXY='https://proxy.golang.org,direct'
GOARCH='arm64'
(…)
```

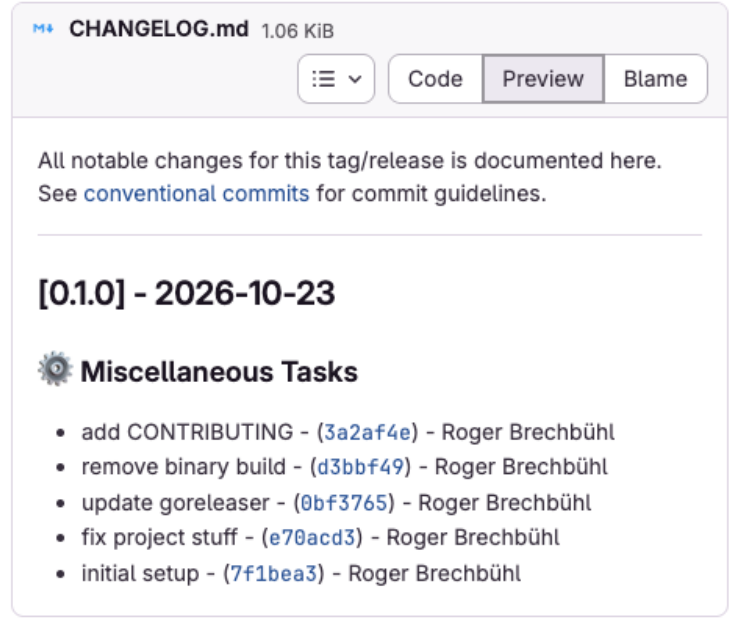
All go env variables are displayed

Golang – “PLATIN’s extensions”

create-go-app is an *internal* developed command-line tool that streamlines Go project setup and management.

- **GoReleaser**: Creates configurations for RPM and Deb packages, Docker Images, Zip archives, and executable binaries – https://goreleaser.com
- **GitLab CI**: Configures comprehensive CI/CD pipelines with modern GitLab CI components support – https://docs.gitlab.com/ci/
- **Taskfile**: Provides extensive local development tasks for building, testing, linting, and debugging – https://taskfile.dev/
- And other features

```
# task changelog
The next version is: v0.1.0. Do you
want to continue? [y/N]: _
```



Golang – “Project layout”

Some rules to consider

- Folder `internal` prevents packages to be used in third party projects, conclusion is that all packages outside of internal are publicly available (*pkg* is just a common practice)
- Packages are imported with folder name, package names should correspond with folder
- Executable go files are always `package main` and contain `func main()`
- Tests resides along with business code

#cmd/jerry/main.go

```
package main

import (
    "fmt"
    "simplego/internal/animals"
)

func main() {
  mouse := &animals.Mouse{}
  fmt.Printf(
    ”Mouse: %s\n", mouse.Sound())
}
```

#internal/animals/mouse.go

```
package animals

type AnimalSounds interface {
    Sound() string
}

type Mouse struct{}

func (m *Mouse) Sound() string {
    return "Squeak"
}
```

```
project/
  go.mod
  cmd/
    jerry/
      main.go
  internal/
    animals/
      mouse.go
      mouse_test.go
  pkg/
    zoo/
      zoo.go
# go mod init project
# go test ./..
# cd cmd/jerry && go run .
# go run cmd/jerry/main.go
```

Golang – “vs Java”

Some major differences between Java and Golang*

- **Paradigm**: Java is class-based OOP with inheritance; Go uses composition, structs, and interfaces (implicit satisfaction, no classes/inheritance).
- **Error handling**: Java uses exceptions (try/catch); Go uses explicit multi-value returns (`val, err :=`) — no exceptions for normal control flow.
- **Concurrency**: Java uses threads, `ExecutorService`, locks/`synchronized`; Go uses lightweight goroutines and channels (CSP model) — much cheaper and more idiomatic concurrency.
- **Runtime/memory**: Java runs on the JVM (bytecode, GC tuning, JIT); Go compiles to a single native static binary with its own lightweight GC — simpler deployment, faster startup, no JVM dependency.

Golang – “Interfaces”

- In Java, interface implementation is ***explicit***: `class Dog implements Animal`
- In Golang, interface implementation is ***implicit***, as soon as a type (e.g. a struct) implements all methods

zoomanager.go

```
func main() {
  cat := &animals.Cat{Name: "Tom"}
  mouse := &animals.Mouse{Name: "Jerry"}
  zoo.FeedAnimals([]animals.Animal{cat, mouse})
}
```

internal/zoo/zoo.go

```
func FeedAnimals(a []animals.Animal) {
  for _, animal := range a {
    fmt.Printf("%s is eating %s\n",
        animal.GetName(), animal.Eat())
    }
}
```

internal/animals/mouse.go

```
type Animal interface {
    Sound() string
    Eat() string
    GetName() string
}

type Mouse struct {
    Name string
}

func (m *Mouse) Sound() string {
    return "Squeak"
}

func (m *Mouse) Eat() string {
    return "Nibble"
}

func (m *Mouse) GetName() string {
    return m.Name
}
```

Golang – “Error handling”

- There is just the “error” interface
- Errors are returned and not thrown
- The caller handle the error in the program flow

```
// handle the error
val, err := convertToInt(“foo”)
if err != nil {
  val = -1
}

// return the error
if err != nil {
  return fmt.Errorf(” ("failed to convert : %w", err)
}

// or ignore the error
val, _ := convertToInt(“bar”)
```

Error handling in Java

- Exception vs Error
- Checked vs Unchecked
- Exception hierarchy
- Try/catch vs throws Exception
- Danger of uncontrolled/complex program flow

```
private int convert(String input) {
  try {
    return Integer.parseInt(input);
  } catch (NumberFormatException e) {
    log.warn("invalid integer value: {}", input);
    // return a default value or rethrow exception?
    return -1;
  }
}
```

Concurrency – “Java”

Just see some code, let’s start with Java….

```
// initialize once
this.mailExecutorService = Executors.newFixedThreadPool(
        Integer.parseInt(Objects.requireNonNullElse(System.getenv(THREAD_POOL_SIZE), "5")),
        new ThreadFactoryBuilder()
                .setNameFormat("alertmanager-v4-mail-pool-%d")
                .setDaemon(true)
                .build()
);

// process messages
payload.getAlerts().stream()
        .filter(x -> x.getResolvedOptions(requestOptions).getMail().isUsable())
        .collect(
                Collectors.groupingBy(x -> x.getResolvedOptions(requestOptions).getMail(), Collectors.toList())
        ).entrySet().parallelStream().forEach(x -> this.mailExecutorService.execute(new MailAlertTask(
                        configuration.getMail(), x.getKey(), x.getValue())
                )
        );
}
```

Concurrency – “Golang”

…then show similar behaviour with Golang:

```
// initialize one
func main() {
  inputQueue := make(chan Message, inputQueueSize)
  go consume(ctx, inputQueue)
  //feed the inputQueue with Messages
}

// consume reads messages from the input channel and processes them.
func consume(ctx context.Context, incoming <-chan Message) {
    for {
       select {
       //omitted ctx handling
       case msg, _ := <-incoming:

          go func(msg Message, parentCtx context.Context) {
             //process the message
          }(msg, ctx)
       }
    }
}
```

Golang linter – “I like…”

- Linters can easily be integrated in IDEs, used on command line and in CI pipelines
- They help to keep a project clean from the beginning
- Issues must be fixed or can be excluded with a meaningful comment

```
# golang-ci-lint run ./...
internal/output/gitlab.go:22:2: G101: Potential hardcoded credentials (gosec)
        gitlabTokenEnv = "EVENTHUB_GITLAB_TOKEN"
```

internal/output/gitlab.go

```
20 (…)
21 // gitlabTokenEnv holds the GitLab API token and overrides the config value.
22 gitlabTokenEnv = "EVENTHUB_GITLAB_TOKEN" // #nosec G101 -- env var name, not a secret
23 (…)
```

Golang linter – “Globar vars, I struggled?”

Phhhh, how to resolve this? Just ignoring is too easy, isn’t it?

```
# golang-ci-lint run ./...
internal/config/config.go:5:5: ZooConfig is a global variable (gochecknoglobals)
var ZooConfig Config
    ^
1 issues:
* gochecknoglobals: 1
```

```
# internal/animals/animals.go
func AnimalConcert(a []animals.Animal) {
  // Use the writer from the global config to output the animal sounds.
  writer := config.ZooConfig.Writer
  for _, animal := range a {
    _, _ = fmt.Fprintf(writer, "%s says %s\n", animal.GetName(), animal.Sound())
  }
}
```

```
# internal/config/config.go
[1] package config
[2]
[3] import "io"
[4]
[5] var ZooConfig Config
[6]
[7] type Config struct {
[8]    Writer io.Writer
[9] }
```

Golang linter – “Globar vars, the solution?”

Finally, zooConfig under control of main, and linter is happy….

internal/animals/animals.go

```
type Zoo struct {
  config config.Config
}

// NewZoo creates a new Zoo instance with the configuration.
func NewZoo(config config.Config) *Zoo {
  return &Zoo{
    config: config,
  }
}

// AnimalConcert simulates a concert with each animals’ sounds.
func (z *Zoo) AnimalConcert(a []animals.Animal) {
  // Use the writer from the zoo configuration.
  writer := z.config.Writer
  for _, animal := range a {
    _, _ = fmt.Fprintf(writer, "%s says %s\n",
                  animal.GetName(), animal.Sound())
  }
}
```

main.go

```
zooConfig := config.Config{
    Writer: os.Stdout,
}

//declaring cat and mouse omitted
theZoo := zoo.NewZoo(zooConfig)
theZoo.AnimalConcert(
         []animals.Animal{cat, mouse})
```

internal/config/config.go

```
[1] package config
[2]
[3] import "io"
[4]
[5] type Config struct {
[6]    Writer io.Writer
[7] }
```

Golang– “Conclusion”

- It feels lightweight, It feels fast
- It works out of the box, e.g. installation and first steps
- It supports what you need nowadays
  - **Tooling**: Testing, Dep Management, Build, Run
  - **Development**: networking, e.g. http, math, string, formatting, io, error, logging, regex and more on the standard library
  - **Building**: Single binary with multiplatform support
- There is a rich ecosystem for extending Golang

References:

- https://go.dev/doc/modules/layout
- https://github.com/golang-standards/project-layout
- https://go.dev/play/

Contact

E-Mail: roger.brechbuehl@postfinance.ch

LinkedIn: https://www.linkedin.com/in/roger-brechbühl-92799632b

It was a pleasure!
Any questions?

PostFinance